Linked Lists

Introduction to Linked Lists

What is a Linked List?

A linked list is a linear data structure where elements, called nodes, are not stored in contiguous memory locations. Instead, each node points to the next node in the sequence using a pointer. The last node's pointer is set to `NULL`, indicating the end of the list.

- **Node Structure:** Each node contains two parts:
  - `data`: Stores the value of the node.
  - `next`: A pointer to the next node in the list.
- **Comparison with Arrays:**
  - **Arrays**: Contiguous memory allocation, fixed size, faster random access.
  - **Linked Lists**: Dynamic size, non-contiguous memory allocation, slower random access, but efficient insertions and deletions.

Node Structure in C

In C, we define a node using a `struct`. Here's the basic structure of a node:

```
struct Node {
    int data;            // Stores the data of the node
    struct Node* next;   // Pointer to the next node in the list
};
```

Dynamic Memory Allocation

Linked lists rely on dynamic memory allocation, typically using `malloc()` to allocate memory for new nodes. After allocating memory, `free()` is used to release it when no longer needed.

```
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = data;
newNode->next = NULL;
```

Creating and Printing a Linked List

```c
#include <stdio.h>
#include <stdlib.h>

// Define the structure of a Node
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to print the linked list
void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = createNode(10);
    head->next = createNode(20);
    head->next->next = createNode(30);

    printList(head); // Output: 10 -> 20 -> 30 -> NULL
    return 0;
}
```

Tasks

- ★ Modify the code to create a linked list with 5 nodes (values: 1, 2, 3, 4, 5).
- ★ Write a function `int length(struct Node* head)` that returns the length of the linked list.

Inserting Nodes

Insertion Operations

- **At the beginning:** Insert a new node before the current head.
- **At the end:** Traverse to the last node and insert the new node.
- **At a given position:** Traverse to the node just before the desired position and insert the new node.

Inserting a Node at the Beginning

```
struct Node* insertAtBeginning(struct Node* head, int data) {
      struct Node* newNode = createNode(data);
      newNode->next = head;
      head = newNode; // New node becomes the head
      return head;
}
```

Tasks

★ Traverse to the last node and insert the new node.

★ Implement a function `insertAtPosition(struct Node* head, int data, int pos)` to insert a node at any given position.

Deleting Nodes

Deletion Operations

- **Deleting the first node:** Set the head to the second node and free the original head.
- **Deleting the last node:** Traverse to the second-last node, set its `next` to `NULL`, and free the last node.
- **Deleting a node at a given position:** Adjust pointers to bypass the node to be deleted, then free it.

Deleting the last node

```
void deleteLastNode(struct Node* head) {
    if (head == NULL) return; // Empty list case
    if (head->next == NULL) { // Only one node case
        free(head);
        head = NULL;
```

```
        return;
    }
    struct Node* temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);  // Free the last node
    temp->next = NULL; // Set second last node's next to NULL
}
```

Task

- ★ Implement a function `deleteAtPosition(struct Node* head, int pos)` to delete a node from any position.
- ★ Delete the middle node from a linked list.
- ★ Remove duplicate elements from an ordered (sorted) Linked List.

Related Concepts (Searching, Reversing, and Loop Detection)

Searching for a Value

A search in a linked list involves traversing the list and comparing each node's data with the target value.

```
int search(struct Node* head, int target) {
    int position = 0;
    struct Node* temp = head;
    while (temp != NULL) {
        if (temp->data == target) {
            return position; // Return the position if found
        }
        temp = temp->next;
        position++;
    }
    return -1; // Return -1 if not found
}
```

Task

- ★ Modify the code to find the k^{th} occurrence of the search key.
- ★ Find the data of the k^{th} node from the end/tail of the linked list.

Reversing a Linked List

Reversing a linked list involves adjusting the `next` pointers of all nodes so that they point to their previous nodes.

Iterative Reversal

```
void reverseList(struct Node* head) {
    struct Node* prev = NULL;
    struct Node* current = head;
    struct Node* next = NULL;

    while (current != NULL) {
        next = current->next;  // Save the next node
        current->next = prev;  // Reverse the pointer
        prev = current;        // Move prev and current one step forward
        current = next;
    }
    head = prev;  // Update the head to point to the new first node
}
```

Task

- ★ Write a recursive function to reverse a linked list.

Loop Detection

Floyd's Cycle-Finding Algorithm (Tortoise and Hare) is used to detect loops in linked lists.

```
int detectLoop(struct Node* head) {
    struct Node* slow = head;
    struct Node* fast = head;

    while (slow != NULL && fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            return 1; // Loop detected
        }
    }
    return 0; // No loop
}
```

Task

- ★ Create a loop in the list and test the loop detection function.
- ★ Remove the loop from the list.

Doubly Linked List

A doubly linked list has nodes with two pointers: one pointing to the next node and one to the previous node.

Node Structure for Doubly Linked List

```
struct DNode {
    int data;
    struct DNode* prev;
    struct DNode* next;
};
```

Inserting at the End of a Doubly Linked List

```
void insertAtEndDoubly(struct DNode* head, int data) {
    struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    struct DNode* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}
```

Task

- ★ Implement a function to delete a node from a doubly linked list.

Circular Linked List

In a circular linked list, the last node's `next` pointer points to the first node, creating a loop.

Traversing a Circular Linked List

```c
void printCircularList(struct Node* head) {
    if (head == NULL) return;

    struct Node* temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(head)\n");
}
```

Tasks

- ★ Create a circular linked list and insert nodes at different positions. Traverse and print the list.
- ★ Modify the circular linked list to handle insertion and deletion at any position.

Homework Tasks

- Sort a linked list.
- Find whether the elements of a linked list form a palindrome. Note → Extra space is not allowed.
